

Cursor AI Prompt Library for Mon Valley Pollution Tracking System Upgrade

Introduction

This document contains specific, copy-paste-ready prompts for Cursor AI to implement each aspect of the application upgrade. These prompts are designed to be used sequentially and provide detailed context for the AI to generate high-quality code.

Foundation Enhancement Prompts

Prompt 1: Project Analysis and Migration Planning

Plain Text

Analyze the current project structure of this React application built with Create React App. Identify:

1. All entry points and main application files
2. Component structure and organization
3. External dependencies and their usage
4. Asset locations (images, fonts, styles)
5. Environment variables and configuration files
6. Build scripts and deployment configuration

Then create a detailed migration plan to move this application to Vite with TypeScript support. List all steps required and potential issues to watch for.

Prompt 2: Vite Migration Execution

Plain Text

Execute the migration from Create React App to Vite:

1. Create a new Vite project with React and TypeScript template in a parallel directory
2. Copy all source files from src/ to the new project
3. Migrate all components to TypeScript, adding proper type definitions
4. Update all import statements to use explicit file extensions where needed
5. Migrate environment variables from REACT_APP_ to VITE_ prefix
6. Update index.html to use Vite's entry point format
7. Configure vite.config.ts with appropriate plugins and settings

8. Update package.json scripts for dev, build, and preview
9. Test that the application runs correctly with 'npm run dev'

Preserve all existing functionality while modernizing the build system.

Prompt 3: Tailwind CSS Integration

Plain Text

Integrate Tailwind CSS into this Vite + React + TypeScript project:

1. Install tailwindcss, postcss, and autoprefixer
2. Create tailwind.config.js with custom theme configuration
3. Create postcss.config.js
4. Add Tailwind directives to the main CSS file
5. Configure the content paths to include all component files

For the custom theme, create a design system inspired by ClimateIQ with:

- A professional color palette with primary (blue-teal), secondary, accent, success, warning, error colors
- Each color should have shades from 50-900
- Typography scale with font sizes from xs to 6xl
- Spacing scale based on 4px units
- Custom shadows for depth
- Border radius values for different component types
- Dark mode support using class strategy

Show me the complete tailwind.config.js file.

Prompt 4: Design Tokens and Theme Setup

Plain Text

Create a comprehensive design system for this application using Tailwind CSS. Define:

1. Color System:

- Primary: Professional blue-teal (#0891B2 base)
- Secondary: Complementary color for accents
- Neutral: Grays for text and backgrounds (50-900)
- Semantic: Success (green), Warning (yellow), Error (red)
- Dark mode variants for all colors

2. Typography System:

- Font families: Sans-serif for UI, Monospace for code
- Font size scale: xs (12px) through 6xl (60px)
- Font weights: light (300), normal (400), medium (500), semibold (600),

bold (700)

- Line heights: tight, normal, relaxed
- Letter spacing for different text sizes

3. Spacing System:

- Base unit: 4px
- Scale: 0, 1, 2, 3, 4, 5, 6, 8, 10, 12, 16, 20, 24, 32, 40, 48, 56, 64

4. Effects:

- Shadow system: sm, md, lg, xl, 2xl
- Border radius: sm, md, lg, xl, 2xl, full
- Transitions: durations and timing functions

Export this as a complete Tailwind config and create a design-tokens.ts file that can be imported in components.

Prompt 5: Base Component Library Structure

Plain Text

Create a reusable UI component library in `src/components/ui/`. For each component, use TypeScript, Tailwind CSS, and follow accessibility best practices.

Create these components:

1. `Button.tsx` - with variants (primary, secondary, outline, ghost), sizes (sm, md, lg), states (default, hover, active, disabled)
2. `Card.tsx` - container with optional header, body, footer sections
3. `Input.tsx` - text input with label, error message, helper text
4. `Select.tsx` - dropdown select with label and error handling
5. `Checkbox.tsx` - checkbox with label and indeterminate state
6. `Modal.tsx` - accessible modal dialog with overlay
7. `Tooltip.tsx` - hover tooltip with positioning

Each component should:

- Accept `className` prop for custom styling
- Be fully keyboard accessible
- Include proper ARIA attributes
- Support dark mode
- Have TypeScript prop types defined

Start with `Button.tsx` and show me the complete implementation.

Animation and Interaction Prompts

Prompt 6: Framer Motion Setup and Page Transitions

Plain Text

Install and configure Framer Motion for this React application. Then implement smooth page transitions:

1. Install framer-motion package
2. Create a PageTransition.tsx component that wraps page content
3. Use AnimatePresence in the main App component to wrap the router
4. Implement fade-in and slide-up animation for page transitions
5. Use these animation values:
 - Duration: 0.3 seconds
 - Easing: easeInOut
 - Initial: opacity 0, y: 20
 - Animate: opacity 1, y: 0
 - Exit: opacity 0, y: -20

Show me the complete implementation including the App component modifications and the PageTransition component.

Prompt 7: Scroll-Triggered Section Animations

Plain Text

Create scroll-triggered animations for content sections using Framer Motion's useInView hook:

1. Create a FadeInSection.tsx component that wraps content
2. Use the useInView hook to detect when the section enters viewport
3. Animate from { opacity: 0, y: 50 } to { opacity: 1, y: 0 }
4. Add a 0.5 second delay between consecutive sections
5. Make the animation trigger once (not on every scroll)

Then apply this component to all major sections on the home page:

- Hero section
- About section
- Mission section
- Features section
- Contact section

Show me the FadeInSection component and example usage.

Prompt 8: Button and Interactive Element Micro-interactions

Plain Text

Add micro-interactions to all interactive elements using Framer Motion:

1. Update the Button component to include:
 - Scale effect on hover (1.02)
 - Scale effect on tap (0.98)
 - Smooth transition (0.2s)
2. Create a LinkHover component for navigation links:
 - Underline animation on hover
 - Color transition
 - Smooth easing
3. Create a CardHover component for card containers:
 - Subtle lift on hover (translateY: -4px)
 - Shadow increase
 - Smooth transition
4. Add loading spinner animation for async operations

Show me the updated Button component and the new hover components.

Prompt 9: Chart and Data Visualization Animations

Plain Text

Add smooth animations to data visualizations:

1. For the PM2.5 trend chart, add:
 - Animate line drawing from left to right on initial load
 - Animate data points appearing with scale effect
 - Smooth transitions when data updates
 - Tooltip fade-in animation
2. For map markers:
 - Bounce animation when markers appear
 - Pulse animation for active/selected markers
 - Smooth transition when markers update
3. For dashboard cards showing statistics:
 - Count-up animation for numbers
 - Fade-in for content
 - Stagger animation for multiple cards

Show me how to implement these animations using Framer Motion and the existing chart library.

Visual Design Overhaul Prompts

Prompt 10: Navigation Bar Redesign

Plain Text

Redesign the main navigation bar to be modern and professional:

1. Create a sticky header that remains at top on scroll
 2. Add blur effect to background when scrolled (backdrop-blur-md)
 3. Replace emoji icons with Lucide React icons:
 - Home: Home icon
 - Dashboard: BarChart3 icon
 - Sensor Map: Map icon
 - Report Symptoms: FileText icon
 - AI Assistant: Bot icon
 - Exposure Risk: AlertTriangle icon
 4. Style the navigation:
 - Dark background with transparency
 - White text with hover effects
 - Smooth transitions
 - Active state indicator
 - Mobile responsive hamburger menu
 5. Add a "Request Demo" button on the right side
- Show me the complete Navigation component with TypeScript and Tailwind CSS.

Prompt 11: Hero Section Implementation

Plain Text

Create a stunning hero section for the home page inspired by ClimateIQ:

1. Full viewport height section
2. Large, bold headline: "Real-Time Air Quality Monitoring for Mon Valley Communities"
3. Subheadline: "Empowering residents with data-driven tools to track pollution and advocate for clean air"

4. Two CTA buttons: "View Dashboard" (primary) and "Learn More" (secondary)
5. Background: High-quality image with dark overlay for text readability
6. Optional: Subtle parallax effect on scroll
7. Scroll indicator at bottom

Use Framer Motion for entrance animations:

- Headline fades in and slides up
- Subheadline follows with slight delay
- Buttons appear last
- Stagger timing: 0.2s between elements

Show me the complete Hero component.

Prompt 12: Color System Implementation

Plain Text

Implement a comprehensive color system throughout the application:

1. Define semantic color usage:
 - Primary: Brand color for key actions and emphasis
 - Secondary: Supporting color for less prominent elements
 - Success: Positive feedback, good air quality
 - Warning: Moderate air quality, caution
 - Error: Poor air quality, errors
 - Neutral: Text, borders, backgrounds
2. Apply colors consistently:
 - Buttons use primary/secondary colors
 - Air quality indicators use success/warning/error
 - Text uses neutral shades based on hierarchy
 - Backgrounds use neutral-50 (light) or neutral-900 (dark)
3. Ensure WCAG AA contrast ratios:
 - Text on backgrounds: minimum 4.5:1
 - Large text: minimum 3:1
 - Interactive elements: minimum 3:1
4. Implement dark mode toggle that switches between light and dark color schemes

Show me the color definitions in `tailwind.config.js` and example usage in components.

Prompt 13: Typography System Implementation

Plain Text

Establish a professional typography system:

1. Define heading hierarchy:
 - h1: 3.75rem (60px), font-bold, leading-tight - for hero headlines
 - h2: 3rem (48px), font-bold, leading-tight - for section titles
 - h3: 2.25rem (36px), font-semibold, leading-snug - for subsections
 - h4: 1.875rem (30px), font-semibold, leading-snug - for card titles
 - h5: 1.5rem (24px), font-medium, leading-normal - for smaller headings
 - h6: 1.25rem (20px), font-medium, leading-normal - for labels
2. Define body text:
 - Large: 1.125rem (18px), leading-relaxed - for important content
 - Base: 1rem (16px), leading-normal - for standard text
 - Small: 0.875rem (14px), leading-normal - for captions
 - Tiny: 0.75rem (12px), leading-tight - for labels
3. Create Typography components:
 - Heading component with level prop (1-6)
 - Text component with size prop (lg, base, sm, xs)
 - Both should support color, weight, and alignment props
4. Apply responsive scaling:
 - Reduce heading sizes by 25% on mobile
 - Maintain readability across all screen sizes

Show me the Typography components and example usage.

Prompt 14: Icon System Integration

Plain Text

Replace all emoji icons with professional icons from Lucide React:

1. Install lucide-react package
2. Create an Icon component that wraps Lucide icons with consistent sizing
3. Replace emoji icons throughout the application:
 - 🏠 → Home
 - 📊 → BarChart3
 - 🗺️ → Map
 - 📝 → FileText
 - 🤖 → Bot
 - 🔬 → Microscope
 - ✓ → Check
 - ✖ → X
 - ⚠️ → AlertTriangle

4. Add icons to buttons, navigation, and feature cards
5. Ensure icons have proper aria-labels for accessibility
6. Support icon sizing (sm: 16px, md: 20px, lg: 24px, xl: 32px)

Show me the Icon component and examples of replacing emojis in the navigation.

Layout and Composition Prompts

Prompt 15: Split-Screen Feature Sections

Plain Text

Create split-screen layouts for the features section:

1. Design a FeatureSection component that:
 - Takes props: title, description, image/icon, imagePosition (left/right)
 - Displays content in a 50/50 split on desktop
 - Stacks vertically on mobile
 - Includes fade-in animation when scrolling into view
2. Implement alternating layouts:
 - First feature: text left, visual right
 - Second feature: text right, visual left
 - Continue alternating
3. Style with:
 - Generous padding and spacing
 - Background color alternation (white, light gray)
 - Responsive images that scale appropriately
4. Create sections for these features:
 - Real-Time Monitoring
 - Source Attribution
 - Health Tracking
 - Evidence Generation
 - AI Assistant
 - Risk Modeling

Show me the FeatureSection component and example usage for all features.

Prompt 16: Full-Width Section Containers

Plain Text

Create a full-width section system for immersive layouts:

1. Create a Section component that:
 - Spans full viewport width
 - Contains content with max-width constraint (1280px)
 - Centers content horizontally
 - Accepts background color/gradient/image
 - Includes padding for vertical spacing
2. Create section variants:
 - Hero: Full viewport height, centered content
 - Content: Standard padding, white background
 - Feature: Large padding, alternating backgrounds
 - CTA: Medium padding, primary color background
 - Footer: Dark background, white text
3. Add props for:
 - variant (hero, content, feature, cta, footer)
 - background (color, gradient, or image URL)
 - padding size (sm, md, lg, xl)
4. Ensure responsive behavior:
 - Appropriate padding on mobile
 - Content max-width adjusts for screen size

Show me the Section component and examples of each variant.

Prompt 17: Card Component System

Plain Text

Create a comprehensive card component system:

1. Base Card component with:
 - Border or shadow for depth
 - Rounded corners
 - Padding for content
 - Optional hover effect (lift and shadow increase)
2. Card variants:
 - Default: White background, subtle shadow
 - Outlined: Border instead of shadow
 - Elevated: Larger shadow, more prominent
 - Interactive: Hover effects, clickable

3. Card sub-components:

- CardHeader: Title and optional actions
- CardBody: Main content area
- CardFooter: Actions or metadata

4. Specialized cards:

- StatCard: Display metric with label and trend
- FeatureCard: Icon, title, description
- DataCard: For dashboard data display

5. Add Framer Motion hover effects:

- Scale: 1.02 on hover
- Shadow: Increase on hover
- Smooth transition

Show me the Card component system with all variants and sub-components.

Prompt 18: Responsive Grid System

Plain Text

Create a responsive grid system for layout flexibility:

1. Create a Grid component that:

- Uses CSS Grid
- Accepts columns prop for different breakpoints
- Supports gap sizing
- Handles responsive behavior automatically

2. Define responsive breakpoints:

- Mobile: 1 column
- Tablet: 2 columns
- Desktop: 3-4 columns
- Wide: 4-6 columns

3. Create grid patterns for:

- Feature cards (3 columns on desktop)
- Dashboard stats (4 columns on desktop)
- Partner logos (5-6 columns on desktop)
- Blog/news items (3 columns on desktop)

4. Add auto-fit and auto-fill options for flexible layouts

5. Ensure proper spacing and alignment

Show me the Grid component and examples for each use case.

Data Visualization Enhancement Prompts

Prompt 19: Custom Leaflet Map Styling

Plain Text

Redesign the Leaflet map with custom styling:

1. Replace default OpenStreetMap tiles with a dark-themed tile layer:
 - Option 1: Mapbox dark theme (requires API key)
 - Option 2: CartoDB dark matter (free)
 - Option 3: Stadia dark theme
2. Create custom map markers:
 - Use React components instead of default Leaflet markers
 - Color-code by air quality level (green, yellow, orange, red, purple)
 - Add pulse animation for active markers
 - Include hover tooltip with sensor details
3. Style map controls:
 - Custom zoom buttons matching app design
 - Layer toggle styled with Tailwind
 - Search box with modern input styling
4. Add map features:
 - Smooth zoom animations
 - Marker clustering for dense areas
 - Custom popup styling when clicking markers

Show me the updated map component with custom styling and markers.

Prompt 20: Animated Chart Components with Recharts

Plain Text

Rebuild the PM2.5 trend chart using Recharts with animations:

1. Install recharts package
2. Create a LineChart component that:
 - Displays PM2.5 data over time
 - Animates line drawing on initial load
 - Shows data points with hover effects
 - Includes custom tooltip with detailed information
 - Uses app color scheme

3. Chart styling:

- Line color: primary color with gradient fill
- Grid: subtle gray lines
- Axes: clean labels with proper formatting
- Responsive: adapts to container width

4. Add features:

- Threshold lines for AQI levels
- Highlighted areas for unhealthy ranges
- Legend for multiple data series
- Export button for data

5. Animation configuration:

- Duration: 1500ms
- Easing: ease-in-out
- Delay: 0ms

Show me the complete chart component with sample data.

Prompt 21: Interactive Tooltips and Popovers

Plain Text

Create rich, interactive tooltips for data visualizations:

1. Build a Tooltip component using Radix UI or Headless UI:

- Positioned automatically (top, bottom, left, right)
- Arrow pointing to trigger element
- Fade-in animation
- Styled with Tailwind CSS

2. Create specialized tooltips:

- MapMarkerTooltip: Sensor name, current AQI, timestamp
- ChartTooltip: Data point value, date, comparison
- InfoTooltip: Help text with icon trigger

3. Add interactive features:

- Hover delay (300ms)
- Click to pin tooltip open
- Close button for pinned tooltips
- Rich content support (text, icons, small charts)

4. Ensure accessibility:

- Proper ARIA attributes
- Keyboard navigation
- Screen reader support

Show me the Tooltip component and examples for map markers and charts.

Prompt 22: Dashboard Statistics Cards

Plain Text

Create animated statistics cards for the dashboard:

1. Build a StatCard component that displays:
 - Large number (the metric value)
 - Label (what the metric represents)
 - Trend indicator (up/down arrow with percentage)
 - Optional sparkline chart
 - Color coding based on value (good/moderate/poor)
2. Add animations:
 - Count-up animation for numbers using Framer Motion
 - Fade-in when card appears in viewport
 - Hover effect (slight lift)
3. Create stat cards for:
 - Current AQI
 - 24-hour average
 - Number of sensors online
 - Symptoms reported today
 - High-risk facilities nearby
4. Arrange in responsive grid:
 - 4 columns on desktop
 - 2 columns on tablet
 - 1 column on mobile

Show me the StatCard component and dashboard layout with all stat cards.

Performance and Polish Prompts

Prompt 23: Route-Based Code Splitting

Plain Text

Implement route-based code splitting to optimize bundle size:

1. Use `React.lazy()` to lazy-load route components:

- HomePage
- DashboardPage
- SensorMapPage
- ReportSymptomsPage
- AIAssistantPage
- ExposureRiskPage

2. Wrap lazy components with Suspense:

- Create a LoadingSpinner component for fallback
- Show loading state while chunk loads
- Add error boundary for chunk load failures

3. Preload routes on hover:

- When user hovers over navigation link, preload that route's chunk
- Improves perceived performance

4. Analyze bundle sizes:

- Use vite-plugin-bundle-analyzer
- Identify large dependencies
- Consider splitting large libraries

Show me the updated routing configuration with lazy loading and Suspense.

Prompt 24: Image Optimization and Lazy Loading

Plain Text

Implement comprehensive image optimization:

1. Create an OptimizedImage component that:

- Lazy loads images below the fold
- Uses Intersection Observer for loading trigger
- Shows skeleton/blur placeholder while loading
- Supports responsive sizes with srcset
- Handles loading errors gracefully

2. Optimize image assets:

- Convert to WebP format with fallbacks
- Generate multiple sizes for responsive loading
- Add proper width and height attributes to prevent layout shift

3. Implement blur-up technique:

- Show tiny blurred placeholder initially
- Fade in full image when loaded
- Smooth transition

4. Add image loading states:

- Skeleton loader
- Progress indicator for large images
- Error state with retry button

Show me the `OptimizedImage` component and usage examples.

Prompt 25: Accessibility Audit and Fixes

Plain Text

Conduct a comprehensive accessibility audit and implement fixes:

1. Install and configure `eslint-plugin-jsx-a11y`
2. Run audit and fix all violations:
 - Add alt text to all images
 - Add labels to all form inputs
 - Ensure proper heading hierarchy
 - Add ARIA attributes where needed
 - Fix color contrast issues
3. Implement keyboard navigation:
 - Ensure all interactive elements are keyboard accessible
 - Add visible focus indicators
 - Implement skip links for main content
 - Add keyboard shortcuts for common actions
4. Test with screen readers:
 - Add proper ARIA labels
 - Use semantic HTML elements
 - Ensure logical tab order
 - Add live regions for dynamic content
5. Create an accessibility statement page

Show me the ESLint configuration and examples of accessibility fixes for key components.

Prompt 26: Performance Monitoring and Optimization

Plain Text

Set up performance monitoring and implement optimizations:

1. Add performance monitoring:
 - Integrate Web Vitals library
 - Track Core Web Vitals (LCP, FID, CLS)

- Log performance metrics to console in development
- Set up analytics reporting for production

2. Optimize React rendering:

- Use React.memo for expensive components
- Implement useMemo for expensive calculations
- Use useCallback for event handlers passed to children
- Identify and fix unnecessary re-renders

3. Optimize data fetching:

- Implement React Query for caching
- Add stale-while-revalidate strategy
- Prefetch data on route hover
- Debounce search inputs

4. Reduce bundle size:

- Remove unused dependencies
- Use tree-shaking effectively
- Dynamically import large libraries
- Analyze and optimize largest chunks

Show me the performance monitoring setup and examples of React optimization techniques.

Prompt 27: Loading States and Skeleton Screens

Plain Text

Create comprehensive loading states and skeleton screens:

1. Build a Skeleton component:

- Animated shimmer effect
- Configurable shapes (rectangle, circle, text line)
- Matches the layout of actual content
- Smooth transition to real content

2. Create skeleton screens for:

- Dashboard cards
- Chart placeholders
- Map loading state
- Table rows
- Form inputs

3. Implement loading patterns:

- Skeleton screens for initial page load
- Inline spinners for button actions
- Progress bars for file uploads

- Shimmer effect for lazy-loaded images

4. Add loading states to all async operations:

- Data fetching
- Form submissions
- Image loading
- Route transitions

Show me the Skeleton component and examples for dashboard and map loading states.

Prompt 28: Error Handling and Error Boundaries

Plain Text

Implement comprehensive error handling:

1. Create an `ErrorBoundary` component:
 - Catches React errors in component tree
 - Shows fallback UI with error message
 - Provides "Try Again" button
 - Logs errors to console (or error tracking service)
2. Create error states for:
 - Data fetching failures
 - Map loading errors
 - Chart rendering errors
 - Form validation errors
 - Network connectivity issues
3. Design error UI:
 - Clear error messages
 - Suggested actions
 - Visual distinction from normal UI
 - Maintain app branding
4. Add error recovery:
 - Retry buttons for failed requests
 - Fallback data when available
 - Graceful degradation
5. Implement toast notifications:
 - Success messages
 - Error alerts
 - Warning notices
 - Info messages

Show me the `ErrorBoundary` component and examples of error handling in data fetching.

Final Integration and Testing Prompts

Prompt 29: Dark Mode Implementation

Plain Text

Implement a complete dark mode system:

1. Set up dark mode with Tailwind:
 - Configure dark mode strategy (class-based)
 - Define dark mode color variants for all colors
 - Create dark mode utilities
2. Create a `ThemeProvider` component:
 - Manages theme state (light/dark/system)
 - Persists preference to `localStorage`
 - Provides theme toggle function
 - Detects system preference
3. Create a `ThemeToggle` component:
 - Button with sun/moon icon
 - Smooth transition animation
 - Accessible label
 - Shows current theme state
4. Apply dark mode styles:
 - Update all components to support dark mode
 - Test contrast ratios in dark mode
 - Ensure images and charts work in both modes
 - Add dark mode variants for map tiles
5. Handle transitions:
 - Smooth color transitions when switching themes
 - No flash of unstyled content
 - Preserve theme across page navigation

Show me the `ThemeProvider`, `ThemeToggle`, and examples of dark mode styling.

Prompt 30: Responsive Design Testing and Fixes

Plain Text

Ensure the application is fully responsive across all devices:

1. Test and fix layouts for these breakpoints:
 - Mobile: 320px - 640px
 - Tablet: 641px - 1024px
 - Desktop: 1025px - 1920px
 - Wide: 1921px+
2. Implement mobile-specific features:
 - Hamburger menu for navigation
 - Touch-friendly button sizes (minimum 44x44px)
 - Swipe gestures for map and charts
 - Collapsible sections for long content
3. Optimize for mobile performance:
 - Reduce image sizes on mobile
 - Simplify animations on low-end devices
 - Defer non-critical resources
4. Test on real devices:
 - iOS Safari
 - Android Chrome
 - Desktop browsers (Chrome, Firefox, Safari, Edge)
5. Fix common responsive issues:
 - Horizontal scrolling
 - Overlapping elements
 - Text too small to read
 - Buttons too small to tap
 - Images not scaling

Show me responsive design fixes for the navigation, hero section, and dashboard.

Usage Instructions

How to Use These Prompts with Cursor AI

1. **Sequential Execution:** Use these prompts in order, as later prompts build on earlier implementations.
2. **Context Provision:** Before using a prompt, ensure Cursor has context of related files by opening them in the editor.

3. **Iterative Refinement:** After Cursor generates code, review it, test it, and provide feedback for refinement.
4. **Customization:** Modify prompts to match your specific requirements, brand guidelines, or technical constraints.
5. **Testing:** After each major change, test the application thoroughly before moving to the next prompt.

Best Practices

- **One Feature at a Time:** Complete each prompt fully before moving to the next.
- **Commit Frequently:** Use version control to save progress after each successful implementation.
- **Review Generated Code:** Always review and understand the code Cursor generates.
- **Test Thoroughly:** Test each feature in different browsers and devices.
- **Document Changes:** Keep notes on what works and what needs adjustment.

Troubleshooting

If Cursor's output doesn't meet expectations:

- Provide more specific requirements in the prompt
 - Share error messages or unexpected behavior
 - Ask Cursor to explain its approach before generating code
 - Break complex prompts into smaller, more focused requests
 - Provide examples of desired output
-

Conclusion

This prompt library provides a comprehensive roadmap for upgrading the Mon Valley Pollution Tracking System using Cursor AI. By following these prompts sequentially and adapting them to your specific needs, you can systematically elevate the application to match the quality and sophistication of ClimateIQ while preserving all existing functionality.

Remember that AI-assisted development is a collaborative process. Review, test, and refine the generated code to ensure it meets your quality standards and functional requirements.